# Create a Sound Sculpture – MUSC 320 Week 7 Assignment

## Overview

In class you heard (and played) a noise sculpture built entirely from MSP signal objects. Now it is your turn. Build your own original sound sculpture in Max/MSP – a patch that produces an evolving, textured sound using the objects you learned this week.

A sound sculpture is not a static tone. It should change and move over time: volumes swell, timbres shift, layers drift in and out. Think of it as a small piece of audio art that rewards a listener who sits with it for 30 seconds or more.

## Requirements

### Signal Objects Checklist

Your patch must use every signal object listed below at least once. Check them off as you go.

**Signal Sources** (use each at least once):

- ☐ `cycle~` – cosine/sine oscillator
- ☐ `noise~` – white noise generator
- ☐ `phasor~` – repeating 0-to-1 ramp

**Signal Operators** (use each at least once):

- ☐ `*~` – signal multiplication
- ☐ `+~` – signal addition
- ☐ `sig~` – number-to-signal conversion
- ☐ `line~` – smooth audio-rate ramps

**Output and Safety:**

- ☐ `live.gain~` – volume control

### Layers

Your patch must have at least **two distinct layers** (also called voices). A layer is an independent signal path with its own source, its own processing, and its own amplitude control.

Look at Patch 06 and Patch 07 from class for examples of multi-layer architecture. In Patch 06, Voice A is a cycle~ with phasor~ tremolo and Voice B is a detuned drone – two separate signal paths summed with +~. Your sculpture should follow the same principle: independent voices combined into one output.

## Suggestions for Exploration

These are optional starting points, not requirements. Try whatever sounds interesting to you.

- **Tremolo:** Use `phasor~` as an LFO to modulate a *~ amplitude control – the volume wobbles at whatever rate you set.

- **Beating:** Run two `cycle~` objects at nearly the same frequency (e.g., 220 and 223 Hz). The slight detuning creates a slow, pulsing beat.
- **Shaped noise:** Feed `noise~` through `*~` driven by a `line~` envelope so the noise fades in and out rather than playing continuously.
- **Modulation depth:** Multiply your `phasor~` LFO by a small number (e.g., `*~ 0.3`) before using it as a modulator. This controls how extreme the effect is.
- **Breakpoint function:** Send `line~` a sequence of target-time pairs (e.g., `0. 0, 1. 500, 0.3 200, 0. 1000`) to create a multi-segment shape – a custom envelope that ramps through several levels instead of just fading in or out.

## Grading Rubric

| Criterion | Excellent (A) | Good (B) | Satisfactory (C) | Incomplete (D/F) |
|---|---|---|---|---|
| **Required Objects (30%)** | All 8 required objects present and used purposefully | All 8 present; one or two used minimally | 6–7 of the required objects present | Fewer than 6 required objects |
| **Multi-Layer Architecture (20%)** | Two or more clearly independent voices with distinct sources, processing, and amplitude control | Two voices present but share some processing or lack independent amplitude control | Attempt at layering, but voices are not clearly separated | Single signal path with no layering |
| **Sound Design & Musicality (25%)** | Sound evolves over time with intentional movement – engages a listener for 30+ seconds | Some evolution; a few parameters change but overall texture is mostly static | Produces sound but feels like a test patch rather than a composition | Static tone or uncontrolled noise |
| **Patch Organization (15%)** | Neat layout with logical signal flow; easy to trace each voice from source to output | Mostly organized; a few crossed cables or unclear sections | Functional but difficult to follow | Tangled or incomprehensible layout |
| **Safety & Output Chain (10%)** | Complete output chain (`live.gain~` and/or `clip~` before `dac~`); no risk of dangerous levels | Output chain present but missing one safety element | Sound reaches speakers but output chain is incomplete | No volume control; risk of dangerously loud output |

## Submission

[Submission instructions TBD]

## Reference

As you build your patch, explore the helpfiles for each object – option-click (Mac) or alt-click (Windows) any object in Max to open its helpfile. The helpfiles include working examples you can learn from.